Compressing Search Spaces for Fast Subgraph Matching via Vertex-Set Simulation

Ben Trovato
Institute for Clarity in Documentation
Dublin, Ireland
trovato@corporation.com

Lars Thørväld
The Thørväld Group
Hekla, Iceland
larst@affiliation.org

Valerie Béranger
Inria Paris-Rocquencourt
Rocquencourt, France
vb@rocquencourt.com

Jörg von Ärbach
University of Tübingen
Tübingen, Germany
jaerbach@uni-tuebingen.edu
myprivate@email.com
second@affiliation.mail

Wang Xiu Ying
Zhe Zuo
East China Normal University
Shanghai, China
firstname.lastname@ecnu.edu.cn

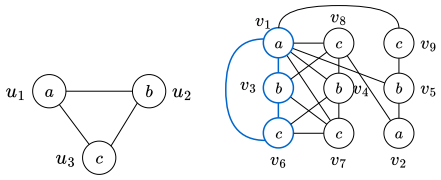
Donald Fauntleroy Duck
Scientific Writing Academy
Duckburg, Calisota
Donald's Second Affiliation
City, country
donald@swa.edu

ABSTRACT

Subgraph matching is a fundamental operation in graph analytics, but its NP-hardness makes exhaustive enumeration expensive, especially on large, dense, and highly symmetric graphs. Most backtracking-based algorithms assign query vertices to data vertices one at a time, which often leads to redundant exploration when many candidates are locally indistinguishable. To reduce such redundancy, we propose VSMatch, a subgraph matching framework that compresses the search space into equivalence classes before enumeration. VSMatch is built on Vertex-set Simulation (VSSim), a structural filtering model that groups candidate vertices with identical neighborhood-support patterns while preserving exact subgraph-isomorphism results. Based on the topological equivalence of VSSim classes, VSMatch performs macroscopic enumeration by mapping query vertices to candidate equivalence classes rather than individual data vertices, thereby reducing the branching factor of the search tree. To further improve enumeration efficiency, VSMatch separates core query vertices from isolated vertices and introduces a label-triggered pre-caching mechanism, which computes valid assignments for isolated vertices once their relevant core constraints are fixed. This allows infeasible branches to be pruned early and avoids expensive tail enumeration through Cartesian-product expansion. Extensive experiments on real-world and synthetically scaled datasets show that VSMatch consistently outperforms state-of-the-art subgraph matching algorithms, achieving improvements of 11 to 18 orders of magnitude in embeddings-per-second(EPS) throughput.

1 INTRODUCTION

Graphs are widely used to model complex relationships among entities in real-world systems. As a fundamental operation in graph analytics, subgraph matching, also known as subgraph isomorphism, has been extensively studied and applied in social network analysis [2, 14, 46], biochemical data processing [8, 13], knowledge graphs [3, 31], fraud detection [16, 42], and graph databases [32]. Given a query graph and a data graph, subgraph matching aims to enumerate all injective mappings from the query graph to the data graph that preserve vertex labels and graph topology. Let us consider the query graph q and the data graph G in Figure 1.



(a) Query Graph q **(b) Data Graph G**

Figure 1: A running example of subgraph matching.

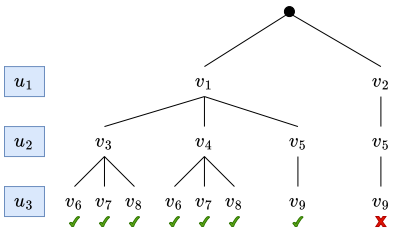


Figure 2: The search tree of traditional backtracking algorithms for the example in Figure 1.

The subgraph induced by the blue vertices in G is a valid match $M : \{(u_1, v_1), (u_2, v_3), (u_3, v_6)\}$.

Despite its broad applicability, subgraph matching is NP-hard [18], and the number of valid embeddings may grow exponentially with the graph size. To improve practical performance, modern exact subgraph matching algorithms typically follow a filtering-ordering-enumeration paradigm [33]. Representative algorithms, such as TurboISO [20], VF3 [10], DPiso [19], CECI [6], BICE [5] have developed effective candidate filtering, matching ordering, and pruning strategies Nevertheless, most of them still rely on a vertex-level backtracking procedure, where each recursive step assigns one query vertex to one candidate data vertex. As illustrated in Figure 2, this point-by-point enumeration can generate many redundant branches when multiple candidate vertices are locally indistinguishable. Such redundancy becomes particularly

severe on large, dense, or highly symmetric graphs, leading to deep recursive search trees and repeated constraint checking.

Several recent methods have attempted to exploit structural redundancy during enumeration. VEQ [30] uses dynamic equivalences to support backward-looking pruning, but it still performs point-by-point backtracking and requires additional conflict-recording structures. IVE [26] separates isolated query vertices from the core matching order, but the delayed handling of isolated vertices may still incur expensive computation at the enumeration tail. B$\mathcal{S}$X [35] groups structurally similar data vertices into search boxes and processes them in batches, but its enumeration remains coupled with a recursive depth-first procedure that repeatedly extracts, refines, and backtracks on boxes at different search depths. These observations suggest that although redundancy-aware techniques are useful, the conventional vertex-level enumeration paradigm still leaves substantial room for search-space compression.

In this paper, we propose *VSMatch*, a novel macroscopic subgraph matching framework that compresses the candidate space into equivalence classes before enumeration. At its core, *Vertex-Set Simulation* (*VSSim*) groups locally indistinguishable candidates while preserving exact matching results. Unlike traditional simulations [15] designed as relaxed metrics, VSSim's monotonic convergence property enables safe class merging and supports *Macroscopic Enumeration*, which maps query vertices to candidate equivalence classes rather than individual data vertices, substantially reducing the branching factor of the search tree. VSMatch further separates core and isolated query vertices and introduces *Label-Triggered Pre-Caching* to compute isolated vertex assignments once their core constraints are fixed. This mechanism prunes infeasible branches early and combines cached assignments through Cartesian products, thereby avoiding expensive tail enumeration.

Contributions. Our main contributions are as follows:

(1) We formulate *Vertex-Set Simulation* (VSSim), a novel graph simulation model. By enforcing macroscopic topological constraints, VSSim serves as a lossless filter and groups locally indistinguishable candidate vertices into equivalence classes.

(2) We formalize the *Candidate Equivalence Space* generated by VSSim and theoretically prove its Completeness, ensuring that transitioning from a vertex-level to a class-level search space introduces zero false negatives, thereby providing a foundation for macroscopic enumeration.

(3) We propose *VSMatch*, an equivalence-class-based subgraph matching framework. Its *Macroscopic Enumeration* algorithm maps query vertices to compressed candidate classes rather than individual data vertices, reducing redundant vertex-level exploration.

(4) We design a label-triggered pre-caching mechanism for isolated query vertices. By computing valid isolated-vertex assignments once their core constraints are fixed, VSMatch prunes infeasible branches early and avoids expensive tail enumeration through Cartesian-product expansion.

(5) We conduct extensive experiments on real-world and synthetically scaled datasets. The results show that VSMatch consistently outperforms state-of-the-art algorithms and achieves up to 11 to 18 orders of magnitude improvement in EPS metric.

Table 1: Summary of notations.

Notation	Description
G, q	The data graph and the query graph
V_G, V_q	The set of vertices in G and q
E_G, E_q	The set of edges in G and q
$L_G(v), L_q(u)$	The label of data vertex v and query vertex u
$N_G(v), N_q(u)$	The set of neighbors of v in G and u in q
$d(v)$	The degree of vertex v, i.e., $\|N(v)\|$
$C(u)$	The candidate set of query vertex u
$[v]$	The equivalence class represented by vertex v
$C_E(u)$	The candidate equivalence classes of query vertex u

2 PRELIMINARIES

Let Γ be a countably infinite set of labels.

Graphs. We focus on undirected labeled graphs $G=(V_G, E_G, L_G)$, where (1) V_G is a finite set of vertices; (2) $E_G \subseteq V_G \times V_G$ is a finite set of edges, where (v, v') is an edge connecting vertex v and v'; and (3) L_G is a labeling function that assigns a label $L_G(v) \in \Gamma$ to each vertex v in V_G.

For each vertex $v \in V_G$, denoted by $N_G(v)$ the set of its neighbors, *i.e.,* $N_G(v)=\{v' \mid (v, v') \in E_G\}$.

Graph pattern matching. We introduce the semantics of subgraph isomorphism and graph simulation. Consider a data graph $G = (V_G, E_G, L_G)$ and a query graph $q = (V_q, E_q, L_q)$. Here, query graph q is also a undirected labeled graph.

Subgraph isomorphism. An isomorphic mapping from q to G is a injective mapping M from V_q to V_G such that (1) for each $u \in V_q$, u and $M(u)$ carry the same label, *i.e.,* $L_q(u)=L_G(M(u))$; and (2) the adjacency relation preserves, *i.e.,* $(u, u') \in E_q$ if and only if $(M(u), M(u')) \in E_G$. When such an injective mapping M exists, we say that q is subgraph isomorphic to G, and denote by $M(q)$ the match of q on G by M, which is an induced subgraph of G by vertex subsets $\{M(u) | u \in V_q\}$.

Graph simulation. A data graph G matches a query graph q via graph simmulation if there exists a binary relation $R \subseteq V_q \times V_G$ such that (1) for each query vertex $u \in V_q$, there exists a vertex $v \in V_G$ such that $(u, v) \in R$; and (2) for each pair $(u, v) \in R$, (a) u and v have the same label, *i.e.,* $L_q(u)=L_G(v)$, and (b) for each query edge (u, u') in E_q, there exists an edge (v, v') in graph G such that $(u, v') \in R$.

It is known that if G matches q via graph simulation, then there exists a unique maximum relation [23], refer to as $Q(G)$.

Example 1: As depicted in Figure 1 and Figure 2, when graph simulation is adopted, the entire data graph G is included in the maximum match relation, which is $\{(u_1, v_1), (u_1, v_2), (u_2, v_3), (u_2, v_4), (u_2, v_5), (u_3, v_6), (u_3, v_7), (u_3, v_8), (u_3, v_9)\}$. □

3 VERTEX-SET SIMULATION

In this section, we define vertex-set simulation, which is designed to group similar candidates while preserving the local structural constraints used by preprocessing for subgraph matching.

Challenges. Although graph simulation is widely used as a light-weight preprocessing technique for subgraph matching, has the following two limitations. (1) When checking whether one vertex v can match a pattern vertex u, graph simulation only inspects the adjacent edges of v and u, and ignores the edges between neighbors of v and u (*i.e.,* triangle relation [37]). Consequently, a vertex in a triangle can match a vertex in a cycle of length 5 (see Example 1). Can we filter such candidates during simulation-based preprocessing? (2) Graph simulation treats candidates individually. Graph simulation only detects whether one vertex v can match a pattern vertex u based on the topological structures of u and v. However, multiple vertices $v_1, v_2, \ldots, v_n$ may be have the same topological structures, and can be exchanged with each other in each match of q. Can we identify such group of vertices during the simulation?

We adopt the following to address these challenges.

(1) We define the binary relations $R' \subseteq V_q \times 2^{V_Q}$, where each element is in the form of (u, V), and V contains all vertices having the same topological structure. To this end, we ensure that all vertices in V have the same adjacent relations with other vertices. For example, given an edge (u_1, u_2), two elements (u_1, V_1) and (u_2, V_2) in R', either all vertices in V_1 have an edge to each vertex in V_2, or none of them has an edge to vertices in V_2. Here V_2 may be the set of candidate matches for another element (u_1, V_1').

(2) To detect the triangle relation, we ensure that given vertices u_1, u_2, u_3 forming a triangle in Q, for each (u_1, V_1), there must exist candidates (u_2, V_2) and (u_3, V_3) such that vertices in V_1, V_2, V_3 form triangles in G.

3.1 Definition of VSSimulation

Vertex-set simulation. Given a query q=(V_q, E_q, L_q) and a graph G=(V_G, E_G, L_G), we say that G matches q via *vertex-set simulation*, denoted by $q \prec_{\text{vs}} G$ if there is a binary match relation $R \subseteq V_q \times 2^{V_G}$ satisfying the following conditions:

(C1) [Same labels] For each $(u, V) \in R$ and each $v \in V$, $L_q(u)$=$L_G(v)$; *i.e.,* each vertex in V has the same label as u;

(C2) [Same neighbors] For each $(u, V) \in R$ and each query edge $(u, u') \in E_q$, (a) there exists $(u', V') \in R$ such that $(v, v') \in E_G$ for each $v \in V$ and $v' \in V'$; and (b) for any $(u', V') \in R$, if there exists $v_0 \in V$ and $v_0' \in V'$ such that $(v_0, v_0') \in E_G$, then each vertex in V has an edge to every vertex $v' \in V'$, *i.e.,* for any $v \in V$ and $v' \in V'$, $(v, v') \in E_G$.

(C3) [Triangles] For each $(u, V) \in R$ and each query edge $(u_1, u_2) \in E_q$, where $u_1, u_2 \in N_q(u)$, there exist $(u_1, V_1), (u_2, V_2) \in R$ such that there are $v \in V$, $v_1 \in V_1$ and $v_2 \in V$ sastisfying (v, v_1), (v, v_2) and (v_1, v_2) are edges in G.

Intuitively, (1) Condition (C1) enforces label consistency between a query vertex and all query vertices in its related vertex subset. (2) Condition (C2) specifies how query edges are preserved between related data vertex sets. It requires that each query edge (u, u') must be preserved by at least one pair of related data vertex sets of u and u', and moreover, once two related data vertex sets contain one cross-set edge, every vertex in one set must be adjacent to every vertex in the other set. (3) Condition (C3) preserves local triangle structures. If two query neighbors of u are also adjacent in the query graph, then their related vertex sets in R must form the corresponding triangle structure in the data graph.

Example 2: Consider the query graph q and data graph G in Figure 1. The query graph q is a triangle with labels a, b, and c.

Consider the data vertex v_1 with label a. To satisfy the structural constraints, v_1 must connect to specific sets for u_2 and u_3 that not only connect to v_1, but also interconnect with each other to mirror the query triangle

In G, v_1 connects to the sets $V_2 = \{v_3, v_4\}$ (label b) and $V_4 = \{v_6, v_7, v_8\}$ (label c). Crucially, these candidate sets fulfill the complete bipartite connectivity mandated by **(C2)**.

Conversely, a vertex like v_2 is filtered out because its only adjacent vertices (v_5 and v_8) lack this necessary cross-connectivity, failing to close the triangle.

By iteratively enforcing these macroscopic constraints, the algorithm converges the valid vertices into structurally cohesive sets, like v_6, v_7, v_8, they share the identical edge structure with their neighbors, and thus are grouped into the same set. The final maximal relation R_{max} is formulated as:

$$R_{max} = \{(u_1, V_1), (u_2, V_2), (u_2, V_3), (u_3, V_4), (u_3, V_5)\}$$

where $V_1 = \{v_1\}$, $V_2 = \{v_3, v_4\}$, $V_3 = \{v_5\}$, $V_4 = \{v_6, v_7, v_8\}$ and $V_5 = \{v_9\}$.

□

While there may be multiple matches in a data graph G for a query graph q, there exists a unique maximum match R_{max} in G for q. To prove this, we first define the topological equivalence relation base on R.

Topological Equivalence Relation. Let $\mathcal{R}$ be a union of valid vertex-set simulation relations. For any query vertex $u \in V_q$, let $\mathcal{V}_u = \{V \mid (u, V) \in \mathcal{R}\}$ denote the family of all valid candidate sets mapped to u. We define a topological equivalence relation $\sim_u$ over $\mathcal{V}_u$. Two sets $V_A, V_B \in \mathcal{V}_u$ are topologically equivalent ($V_A \sim_u V_B$) if and only if they share strictly identical connectivity toward the mapped sets of all adjacent query vertices, *i.e.,* $\forall u_n \in N_q(u), \forall V_n \in \mathcal{V}_{u_n}$

$$V_A \times V_n \subseteq E_G \iff V_B \times V_n \subseteq E_G.$$

Here, the Cartesian product constraint $V \times V_n \subseteq E_G$ formally enforces the complete bipartite connectivity(*i.e.,* $\forall v \in V, v' \in V_n, (v, v') \in E_G$) mandated by **(C2)**.

Property. A vertex-set simulation R is maximal, if there do not exist another vertex-set simulation R', a pattern vertex u and vertex v such that $R \subset R'$, $(u, V') \in R' \setminus R$ with $v \in V'$ and no $(u, V_1) \in R$ satisfies $v \in V_1$. The condition $R \subset R'$ is not sufficient to characterize the maximum of vertex-set simulation, since given an element $(u, V) \in R$, we can add (u, V') with $V' \subseteq V$ into R and show that the resulting relations also satisfy the three condition for the vertex-set simulation. Intuitively, we require that the new relation R' has a new match $(u, \{v\})$ that has not in R.

Theorem 1: *For a query q and data graph G with $q \prec_{\text{vs}} G$, there is a unique minimum maximal vertex-set simulation R in G for q.* □

Proof. We prove it by contradiction. Assume that R_1 and R_2 are two distinct minimum complete vertex-set simulation relations. We define a new vertex-set simulation relations $\mathcal{R}$, and show that $\mathcal{R}$ is complete and $|\mathcal{R}| < |R_1|$ and $|\mathcal{R}| < |R_2|$, a contradiction.

We define $\mathcal{R}$ as follows: for each vertex u in q and each vertex v in G, if there exist $(u, V_v^1) \in R_1$ and $(u, V_v^2) \in R_2$ satisfying $v \in V_v^1 \cap V_v^2$, then $\mathcal{R}$ contains $(u, V_v^1 \cup V_v^2)$.

We next show that (a) $\mathcal{R}$ is a vertex-set simulation; (b) $\mathcal{R}$ is complete; and (c) $|\mathcal{R}| < |R_1|$ and $|\mathcal{R}| < |R_2|$.

Before showing the results, we first establish a property for the maximal vertex-set simulation. Given two maximal vertex-set simulations R_1 and R_2, for each $(u, V_1) \in R_1$ and each $v \in V_1$, there exists an element $(u, V_2) \in R_2$ such that $v \in V_2$. We prove it by contradiction. Assume that there exists an element $(u, V_1) \in R_1$ and a vertex $v \in V_1$, there does not exist $(u, V_2) \in R_2$ satisfying that $v \in V_2$. We can show that R_2 is not maximal. To this end, we extend R_2 by splitting R_1. More specifically, for each element $(u, V) \in R_1$ we add $(u, \{v\})$ to R_2 for all $v \in V$. Since R_1 is a vertex-set simulation and $\{v\}$ consists of only one vertex, we can verify that the resulting relation is also a vertex-set simulation. Because there does not exist $(u, V_2) \in R_2$ satisfying that $v \in V_2$, we can conclude that R_1 is not maximal, a contradiction.

(a) We first show that $\mathcal{R}$ is a vertex-set simulation. We prove that $\mathcal{R}$ satisfies the three conditions for the definition of vertex-set simulation. (i) Because $v \in V_v^1 \cap V_v^2$, all vertices in $V_v^1 \cup V_v^2$ carry the same label as v and u. So $L_q(u) = L_G(v)$ for each $v \in V_v^1 \cup V_v^2$; (ii) for each query edge $(u, u') \in E_q$, given $(u, V_1) \in \mathcal{R}$ and $(u', V_2) \in \mathcal{R}$, if there exists an edge from $v_1 \in V_1$ to $v_1 \in V_2$, we show that there exists an edge from v to v' for each vertex $v \in V_1$ and $v' \in V_2$. Indeed, assume that V_1 is constructed by merging $V_1^v, \ldots, V_k^v$ and V_2 is constructed by merging $V_1^{v_2}, \ldots, V_k^{v_2}$. Moreover, v_1 (resp. v_2) must exist in some set V_j^v (resp. $V_k^{v_1}$). Because R_1 and R_2 are vertex-set simulation, all these vertices having the edges from $V_1^{v_2}, \ldots, V_k^{v_2}$ to V_j^v (resp. $V_k^{v_1}$). Therefore, the results holds. For the triangle relations, R_1 and R_2 are vertex-set simulation, there must exists the required edges.

(2) We next show that (b) $\mathcal{R}$ is maximal. We prove it by contradiction. If there exists another vertex-set simulation R' such that $R \subset R'$, $(u, V') \in R' \setminus R$ with $v \in V'$ and no $(u, V_1) \in R$ satisfies $v \in V_1$. We can prove that both R_1 and R_2 is not maximal, a contradiction. Indeed, we can extend R_1 and R_2 using the matches $(u, \{v\})$.

(3) We finally show that $|\mathcal{R}| < |R_1|$ and $|\mathcal{R}| < |R_2|$. Indeed, since R_1 and R_2 are two two maximal vertex-set simulations. There exist four vertex sets are merged into not sets. That is, $\mathcal{R}| < |R_1|$ and $|\mathcal{R}| < |R_2|$. □

3.2 An Algorithm for vetex-set simulation

Given pattern Q and graph G, we next develop an algorithm to compute the minimum maximal vertex-set simulation VSSim. The algorithm has the following three phases:

Phase 1: Initialization. The algorithm first group vertices having the same labels and neighborhoods (lines 1-3). For each pattern vertex u, it first collects all vertices in G having the same label as u (Line 1). After that, it partitions vertex set $C(u)$ for each pattern vertex u into multiple groups based on their adjacent edges (line 2). More specifically, two vertices v_1 and v_2 in $C(u)$ are in the same group when the following condition holds: for each pattern edge (u, u') and each vertex $v' \in C(u')$, (v_1, v') is an edge in G if and only if (v_1, v') is an edge in G.

Input: Query graph q, Data graph G.
Output: The maximum vertex-set simulation relation R_{max}.

```
1:  C := Group(Q, G)
2:  P := Partition(Q, G, C);
3:  changed ← true
4:  while changed do
5:      changed ← false
6:      for each u ∈ V_q and each K ∈ P(u) do
7:          bExistEdge := checkAdj(Q, G, u, K);
8:          bTriangle := checkTri(Q, G, u, K);
9:          if bExistEdge = false ∨ bTriangle = false then
10:             P(u) := P(u) \ {K};
11:             changed ← true;
12:         end if
13:     end for
14: end while
15: R_max := ∅;
16: for each query vertex u ∈ V_q do
17:     EC(u) : MergeSet(Q, G, P(u));
18:     for each equivalence class K_eq ∈ EC(u) do
19:         R_max := R_max ∪ {(u, K_eq)}
20:     end for
21: end for
22: return R_max
```

Figure 3: Algorithm VSSim

Phase 2: Iterative Constraint Enforcement. The algorithm removes groups that do not satisfy the second and third conditions for the vertex-set simulation VSSim (lines 3-11). More specifically, for

The algorithm initiates constraint convergence via a **repeat-until** loop (Line 7-14). In (Lines 10-11), the algorithm reduces the vertex-level edge constraint and triangular tests into macroscopic set-based evaluations. If a specific class K fails to find a supporting target class to fulfill its complete bipartite structure or local tripartite cycles, it is pruned as an entire entity in (Lines 12-14). This iteration persists until no further classes are invalidated, strictly reaching a stable fixed point.

Phase 3: Maximal Agglomeration. After the invalid topological structures are purged, the algorithm constructs the unique maximum relation R_{max}. In (Lines 18-19), it formulates the topological equivalence relation $\sim_u$ and computes the quotient set in (Line 20), identifying the classes that share perfectly identical structural connectivities. Subsequently, in (Lines 21-23), the algorithm performs a component-wise union, collapsing these equivalent classes into

super-sets V_{merged} and constructing R_{max}. The final R_{max} returned in (Line 24).

Example 3: To intuitively illustrate the execution workflow of Algorithm 3, we trace its execution using the query graph q and data graph G depicted in Figure 1.

(1) In **Phase 1**, we get the initial candidate sets by label: $C(u_1) = \{v_1, v_2\}$, $C(u_2) = \{v_3, v_4, v_5\}$, and $C(u_3) = \{v_6, v_7, v_8, v_9\}$. Then we partition these candidates into initial classes based on their connective projections. For instance, for u_2, candidates v_3 and v_4 share identical adjacency patterns towards the candidate sets of u_1 and u_3, *i.e.,* $N(v_3) \cap C(u_1) = N(v_4) \cap C(u_1) = \{v_1\}$ and $N(v_3) \cap C(u_3) = N(v_4) \cap C(u_3) = \{v_6, v_7, v_8\}$. Thus they are grouped into the same class $K_{21} = \{v_3, v_4\}$. However, candidate v_5 is placed in a separate class $K_{22} = \{v_5\}$ for its distinct adjacency patterns: $N(v_5) \cap C(u_1) = \{v_1, v_2\}$ and $N(v_5) \cap C(u_3) = \{v_9\}$. So, $\mathcal{K}_{u_2} = \{K_{21}, k_{22}\} = \{v_3, v_4, v_5\}$. In the same way, we can get $\mathcal{K}_{u_1} = \{\{v_1\}, \{v_2\}\}$ and $\mathcal{K}_{u_3} = \{\{v_6, v_7\}, \{v_8\}, \{v_9\}\}$.

(2) In **Phase 2**, the algorithm evaluates the macroscopic constraints iteratively. Consider the class $\{v_2\}$, its only adjacent target classes are $\{v_5\}$ and $\{v_8\}$. However, **(C3)** reveals that $\{v_5\}$ and $\{v_8\}$ are not connected to each other in G. Thus, the classes $\{v_2\}$, $\{v_5\}$, and $\{v_8\}$ fail to formulate the complete tripartite graph required for the query triangle. Consequently, the class $\{v_2\}$ is pruned from $\mathcal{K}_{u_1}$.

(3) Finally, in **Phase 3**, the algorithm converges the surviving valid classes to construct R_{max}. The topological equivalence relation $\sim_u$ dictates the merging process. Because $K_{31} = \{v_6, v_7\}$ and $K_{32} = \{v_8\}$ connect to the same classes of $u_n \in u_3$ (*i.e.,* connects to $\{v_3, v_4\} \in \mathcal{K}_{u_3}$ and $\{v_1\} \in \mathcal{K}_{u_1}$ they are merged into same quotient sets. The algorithm naturally outputs the maximal relation R_{max} as a set of fully compressed, disjoint equivalence classes:

$$R_{max} = \{(u_1, V_1), (u_2, V_2), (u_2, V_3), (u_3, V_4), (u_3, V_5)\}$$

where $V_1 = \{v_1\}$, $V_2 = \{v_3, v_4\}$, $V_3 = \{v_5\}$, $V_4 = \{v_6, v_7, v_8\}$, and $V_5 = \{v_9\}$. This output flawlessly provides the exact boundaries required for subsequent macroscopic enumeration. □

Correctness. Given a query graph q and a data graph G, Algorithm 3 correctly computes the unique maximum vertex-set simulation relation R_{max}.

Proof. In Phase 1, the initialization filters vertices using $L_q(u) = L_G(v)$, guaranteeing the label constraint **(C1)**. Furthermore, the fine-grained partitioning groups vertices with identical connective projections, pre-conditions the classes to obey the complete bipartite connectivity required by **(C2)**.

In Phase 2, the iteration enforces this existential validity. It guarantees that for every surviving class $K \in \mathcal{K}_u$, there strictly exists a surviving target class K_n to fulfill the edge constraint for **(C2)**, alongside the tripartite cyclic structures mandated by **(C3)**.

In Phase 3, the algorithm merges these surviving classes to construct R_{max}. Crucially, the merging is mathematically restricted by the topological equivalence relation $\sim_u$. As proven in Proposition **??**, computing the component-wise union exclusively over $\sim_u$-equivalent classes strictly preserves all macroscopic connectivities. Therefore, R_{max} rigorously satisfies all constraints and constitutes a valid vertex-set simulation. □

Complexity Analysis. Let d_G be the maximum degree of the data graph G, and $\mu_K = \max_{u \in V_q} |\mathcal{K}_u|$ denote the maximum number of candidate classes for any query vertex after initialization.

In Phase 1, the label filtering and connective projection partitioning process each data vertex and its neighborhood, taking $O(|V_q| \cdot |V_G| \cdot d_G)$ time. In Phase 2, let I be the number of iterations required to reach the fixed point. Within each iteration, evaluating the macroscopic complete bipartite **(C2)** and tripartite **(C3)** constraints involves checking interactions between classes rather than individual vertices. This class-level verification takes $O(|E_q| \cdot \mu_K^3)$ time per iteration. In Phase 3, computing the quotient sets via the topological equivalence relation $\sim_u$ across all neighbors requires $O(|E_q| \cdot \mu_K^2)$ time.

Therefore, the overall time complexity of Algorithm 3 is bounded by $O(|V_q||V_G|d_G + I \cdot |E_q| \cdot \mu_K^3)$. Crucially, because vertices with identical local topologies are packed into macroscopic classes during Phase 1, our framework guarantees that $\mu_K \ll |V_G|$. By substituting the vertex space with compressed class boundaries, this algorithm fundamentally bypasses the expensive vertex-by-vertex combinatorial verification overhead that plagues traditional structural simulations.

3.3 Bridging VSSIM to Exact Enumeration

While the maximal relation R_{max} establishes the theoretical boundary for vertex-set simulation, exact subgraph enumeration algorithms require concrete search spaces rather than abstract binary relations. To bridge our filtering theory with the downstream exact matching pipeline, we formalize R_{max} into a compact, class-level search space.

In the preceding definitions, we utilized the generic set notation V to denote a valid subset mapped to a query vertex u. As established in Proposition **??**, the maximal relation R_{max} partitions the candidate space into disjoint quotient sets governed by the topological equivalence relation $\sim_u$. To reflect this structural property, we transition our notation from V to the standard equivalence class symbol $[v]$. An equivalence class $[v]$ encapsulates data vertices that share identical macroscopic connectivity with respect to the query structure.

Building upon this refined notation, we define the *Candidate Equivalence Space* for a query vertex $u \in V_q$ as the image of u under R_{max}:

$$C_E(u) = R_{max}(u) = \{[v] \in 2^{V_G} \mid (u, [v]) \in R_{max}\}$$

This formulation translates the filtering results into a data structure for enumeration. Instead of yielding a flat pool of individual vertices, VSSim outputs $C_E(u)$ as a set of disjoint equivalence classes. We guarantee that any exact subgraph isomorphism must map u to exactly one vertex within a specific class $[v] \in C_E(u)$. Consequently, the downstream enumeration algorithm can navigate the search tree at the class level, processing entire symmetric groups $[v]$ as single units. This macroscopic branching significantly reduces the combinatorial overhead caused by dense local symmetric structures.

However, the fundamental prerequisite for utilizing C_E as an enumeration search space is ensuring thar VSSim does not eliminate any valid exact matches. We formalize this safeguard through the following completeness property.

Input: Data graph $G = (V_G, E_G)$, query graph $q = (V_q, E_q)$.
Output: Set $\mathcal{M}$ of matches of q on G.

$\mathcal{M} \leftarrow \emptyset$
$C_E \leftarrow \text{VSFilter}(q, G)$
$order \leftarrow \text{Ordering}(q, G, C_E)$
$\mathcal{M} \leftarrow \text{VSEnum}(q, G, \mathcal{M}, order, C_E)$
return $\mathcal{M}$

Figure 4: Algorithm VSMatch

Proposition 2: ***Completeness of the Candidate Equivalence Space.*** *Let M be a valid subgraph isomorphic mapping from query graph q to data graph G. For every query vertex $u \in V_q$, its mapped data vertex $M(u)$ is contained within exactly one equivalence class $[v] \in C_E(u)$.* □

Proof. By the definition of subgraph isomorphism, M preserves label equivalence and edge topology. Consider the localized relation $R_M = \{(u, \{M(u)\}) \mid u \in V_q\}$. R_M satisfies all vertex-set simulation constraints: **(C1)** holds via label preservation; **(C2)** and **(C3)** hold because M ensures the existence of corresponding edges $(M(u), M(u_n))$ and triangular closures for every query adjacency. Thus, R_M is a valid vertex-set simulation relation.

As proven in Proposition **??**, R_{max} is the unique maximal relation that subsumes all valid simulation relations. Therefore, $R_M \sqsubseteq R_{max}$. This implies that for every mapping $(u, \{M(u)\}) \in R_M$, there exists an equivalence class $[v]$ in the unified image $R_{max}(u)$ such that the singleton set $\{M(u)\} \subseteq [v]$. Since $R_{max}(u) = C_E(u)$ by definition, the matching vertex $M(u)$ is safely preserved within an equivalence class in $C_E(u)$. □

4 ALGORITHMS FOR VSMatch

In this section, we present VSMatch framework and its algorithmic implementation for subgraph matching under subgraph isomorphic semantics. Based on the VSSim proposed in Section 3, VSMatch reduces redundant exploration by operating on vertex sets rather than individual candidate vertices.

The corresponding execution procedure is summarized in Figure 4, which invokes three algorithms to implement the VSSim filtering, adaptive ordering and macroscopic enumeration in sequence. The detailed algorithms for these procedures are presented in Sections 4.1, 4.2 and 4.3, respectively.

4.1 Macroscopic Filtering via Bounded VSSim

The first phase of our framework aims to materialize the Candidate Equivalence Space C_E by filtering the data graph. While Algorithm 3 establishes the unique maximal relation R_{max} by iterating until a strict fixed point is reached, achieving this absolute fixed point can introduce diminishing returns in filtering overhead.

To strike an balance between the filtering cost and the pruning effectiveness, VSMatch implements a *Bounded VSSim Filter*. Specifically, we execute Phase 1 (Initialization and Class Partitioning) exactly as defined in Algorithm 3. However, during Phase 2 (Iterative Constraint Enforcement), we terminate the constraint validation loop after a small constant number of iterations (denoted as $iters$), rather than waiting for absolute convergence. In our empirical evaluations, setting $iters = 2$ efficiently eliminates the vast majority of invalid structural branches.

Input: Query graph q, data graph G, candidates set C_E
Output: Isolated vertices V_q^I, Core matching order $order$, Label triggers $\mathcal{T}_L$

Initialize $\mathcal{T}_L \leftarrow \emptyset$
$V_q^I \leftarrow \text{GenerateIsolated(q)}$;
for each $u \in V_q^I$ **do**
  if $u \in V_q^{core}$ **then**
    $trig(u) \leftarrow u$;
  else
    $trig(u) \leftarrow \arg\max_{v \in (N_q(u) \cap V_q^{core})} (\text{index of } v)$
  end if
end for
for each distinct label l present in q **do**
  Let V_l be the set of query vertices with label l
  $T_L(l) \leftarrow \arg\max_{u \in V_l} (\text{index of } trig(u))$
  $\mathcal{T}_L \leftarrow \mathcal{T}_L \cup \{(l \rightarrow T_L(l))\}$
end for
return $order, \mathcal{T}_L, V_q^I$

Figure 5: Algorithm Ordering

Crucially, this engineering optimization does not compromise the correctness of our framework. Throughout the filtering process, the surviving equivalence classes consistently maintain the complete bipartite connectivity mandated by **(C2)**. Because the algorithm only eliminates invalid candidates without disrupting the underlying topological equivalence, halting the iteration early naturally yields a relation that is a *superset* of the maximum relation R_{max}. Consequently, the bounded candidate space produced by this phase fundamentally satisfies the **Completeness (Proposition 2)**. Every valid subgraph matching embedding is safely preserved within the extracted class boundaries, ensuring zero false negatives while accelerating the preprocessing stage.

4.2 Ordering and Build Label Trigger

After vertex-set simulation filtering, VSMatch constructs ordering information used in enumeration. Unlike conventional subgraph matching methods that only build a linear matching order, VSMatch constructs three objects, including an isolated vertex set, a core matching order, and label triggers for pre-caching.

Existing methods such as IVE [26] have shown that separating isolated vertices from the core matching order can reduce the depth of enumeration. Following this idea, VSMatch separates isolated vertices from the core matching order, but handles them differently. Instead of postponing all isolated-vertex computation to the enumeration tail, VSMatch builds a label trigger along the core matching order and invokes pre-caching when the corresponding trigger is reached. This design allows part of the delayed computation to be performed earlier, so infeasible branches to be pruned earlier and cached partial mappings to be reused in the final Cartesian-product expansion.

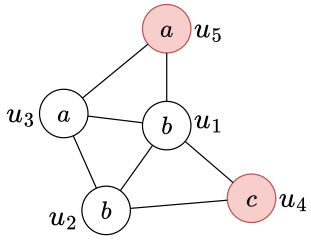


Figure 6: An example of the Label-Trigger Mechanism. Vertices u_4 and u_5 (highlighted in red) are identified as isolated vertices.

Construction of the core matching order. For the query graph $q = (V_q, E_q, L_q)$, the *isolated vertices* of q, denoted by V_q^I, is a set of vertices in q that are pairwise non-adjacent whose removal does not disconnect the remaining query graph. That is, there is no edge in q between any two vertices in V_q^I, and the induced subgraph of vertices in $V_q \setminus V_q^I$ remains connected.

The remaining vertices in $V_q \setminus V_q^I$, forming the *core vertices* of q, are used to construct the core matching order $Order$ as follows. Given the candidate sets C_E after the vertex-set simulation filtering, the core vertices u are ordered in ascending order of $|C_E(u)|$, where $|C_E(u)|$ is the number of candidate equivalence classes of u. This ordering places vertices with fewer candidate classes earlier and therefore reduces the branching factor of the core enumeration.

One would select a maximum isolated vertex set to minimize the size of the core matching order. Since finding such a set exactly may introduce non-negligible overhead, VSMatch constructs V_q^I using a greedy strategy used in IVE[26].

Construction of label triggers. After the core matching order is determined, we construct label triggers for pre-caching.

For each query vertex u in V_q, we first define its *trigger point*, denoted by $trig(u)$ as follows. If u is a core vertex, then its trigger point is itself, *i.e.,* $trig(u) = u$ for each $u \in V_q \setminus V_q^I$; otherwise, u is an isolated vertex, then $trig(u)$ is defined as the neighbor of u that appears latest in the core matching order. Intuitively, when the enumeration reaches $trig(u)$, all core neighbors of the isolated vertex u have already been fixed. Therefore, candidate assignments for u can be checked against the current partial core mapping according to the subgraph isomorphism definition.

For each label l appearing in q, we next defines a *label trigger* $T_L(l)$ of l as the latest trigger point among all query vertices with label l. We denote by $\mathcal{T}_L$ the set of all label triggers. The label triggers are used later during macroscopic enumeration. When the enumeration reaches a core vertex corresponding to $T_L(l)$, VSMatch invokes pre-caching for the isolated vertices with label l under the current partial core mapping. The detailed pre-caching procedure and its integration with macroscopic enumeration are presented in Section 4.3.

Example 4: Consider the query graph topology in Figure 6, where u_4 and u_5 are isolated vertices. Assume that candidate set sizes dictate a core matching order of $\{u_1, u_2, u_3\}$. The triggers are computed as follows: For the isolated vertex u_4, its non-isolated neighbors are u_1 and u_2. Since u_2 appears later than u_1 in the core order, $trig(u_4) = u_2$. Similarly, for u_5, its trigger is $trig(u_5) = u_3$. For the core vertices, $trig(u_1) = u_1$, $trig(u_2) = u_2$, and $trig(u_3) = u_3$. Next, we aggregate these to form the Label Triggers T_L. For label a (vertices u_3, u_5), $T_L(a) = \max_{order}(trig(u_3), trig(u_5)) = u_3$. For label b (vertices u_1, u_2), $T_L(b) = \max_{order}(trig(u_1), trig(u_2)) = u_2$. For label c (vertex u_4), $T_L(c) = trig(u_4) = u_2$. □

4.3 Enumeration Process

After the filtering and matching-order construction, VSMatch performs enumeration over equivalence classes. Unlike conventional backtracking algorithms that assign one query vertex to one data vertex at each recursive step, VSMatch enumerates mappings from core query vertices to candidate equivalence classes. The isolated vertices are not recursively expanded at the tail of the search tree; instead, their valid assignments are precomputed when the corresponding label triggers are reached and are combined with the completed core match through Cartesian products.

Given the equivalence-class candidate sets C_E, the matching order $order$, the isolated vertex set V_q^I, and the label triggers $\mathcal{T}_L$, We develop Algorithm VSEnum (see Figure 7) for the enumeration procedure, which maintains a partial set-level mapping M_p for the core vertices to equivalence class and a Global_Cache which store the partial embedding for each label l.

During the core enumeration, label triggers determine when pre-caching is invoked. When the current core vertex reaches $T_L(l)$ for some label l, VSMatch computes valid assignments for the isolated vertices with label l under the current partial core mapping. If no valid assignment exists, the current branch is infeasible and can be pruned immediately. Otherwise, the computed assignments are stored in the Global_Cache associated with the current branch. These cached assignments are reused after the core sequence has been fully matched.

The algorithm is structured around three core concepts that drive its efficiency:

(1) Macroscopic Class-Level Branching: Instead of unfolding the search tree point-by-point, our enumeration branches out via equivalence classes $[v]$ (Line 13). During this expansion, the function ValidateClassConstraints (Line 14) acts as a lightweight injectivity guard: it specifically monitors singleton classes (i.e., $|[v]| = 1$), instantly pruning the branch if its sole data vertex is already occupied in M_p, and tentatively accepting it otherwise. The rigorous, fine-grained multi-vertex conflict resolution is strategically deferred to the ComputePartialEmbeddings execution. This macroscopic mapping (Line 15) reduces the branching factor of the search space from $O(|C(u_i)|)$ down to $O(|C_E(u_i)|)$, granting our framework extreme resilience against highly symmetric and dense data graphs. Line 16 refines the candidate classes sets, updates the candidate classes sets of the neighboring vertices of u_i in the unenumerated vertices. For each unenumerated neighbor $u_n \in N_q(u_i)$, refine $C_E(u_n)$ as $C'_E(u_n) = \{[v'] \in C_E(u_n) | v_{rep} \in N(v')\}$, where v_{rep} is the representative vertex of $[v]$. For other vertices $C'_E(u) = C_E(u)$.

(2) Conflict-Aware Partial Embedding Computation: As the algorithm explores the core sequence, it actively monitors for Label

Input: Query q, Data G, Partial match M_p, Candidate class sets C_E, Order $order$, Isolated vertices V_q^I, Label triggers $\mathcal{T}_L$
Output: All valid matches $\mathcal{M}$

▷ Bypassing Tail Explosion via Cartesian Product
if AllCoreVerticesMatched(M_p) **then**
  return CartesianProduct(Global_Cache)
end if
$u_i \leftarrow$ GetNextVertex(order)
▷ Label-Triggered Pre-Caching Execution
if u_i is a Trigger Point for some label l **then**
  local_matches←ComputePartialEmbeddings(l, C_E, M_p)
  if local_matches is empty **then**
    return $\emptyset$ ▷ Global Dead-end
  end if
  Global_Cache.Store(l, local_matches)
end if
▷ Class-Level Macroscopic Branching
$\mathcal{M} \leftarrow \emptyset$
for each equivalence class $[v] \in C_E(u_i)$ **do**
  if ValidateClassConstraints($[v], M_p$) **then**
    $M'_p \leftarrow M_p \cup \{(u_i, [v])\}$
    $C'_E \leftarrow$ RefineCandidateClasses(C_E, $[v]$)
    $\mathcal{M} \leftarrow \mathcal{M} \cup$ VSEnum($q, G, M'_p, C'_E, order, V_q^I, \mathcal{T}_L$)
  end if
end for
return $\mathcal{M}$

Figure 7: Algorithm VSEnum

Triggers. When the label trigger of label l is hit (Line 6), all topological constraints for isolated vertices associated with l are fully bounded by M_p. The ComputePartialEmbeddings is then invoked to compute the valid partial matchings for all query vertices with label l.

At this stage, the candidate pool for each non-isolated vertex is restricted to the vertices in its assigned equivalence class in M_p, while that of each isolated vertex consists of all vertices within its valid candidate classes refined with M_p. We propose a *Conflict-Aware Partitioning* strategy. By globally tracking the occurrence frequency of each data vertex across these pools, we partition the candidate data vertices into two disjoint sets: *Potential Conflict Vertices* (frequency > 1) and *Conflict-Free Vertices* (frequency = 1).

We simply enumerate all possible partial matchings for the potential conflict vertices and check for vertex conflicts. Subsequently, for the conflict-free candidate vertices, we can directly obtain the complete local embeddings by applying a Cartesian product.

If no valid local embedding can be formed, the entire subtree rooted at M_p is instantly pruned (Line 9). Otherwise, the pristine local embeddings are stored in the Global_Cache.

(3) Bypassing Tail Enumeration via Cartesian Product: When the core sequence is fully mapped (Line 2), the search tree reaches its maximum required depth. Unlike previous approaches (e.g., IVE) that resolve the remaining isolated vertices via delayed bipartite graph matching at the tail, our algorithm completely bypasses this redundant stage.

Recall that in Line 6-10, the ComputePartialEmbeddings has already resolved local injectivity conflicts and accurately calculated the total number of valid partial matchings for each label, storing these combinatorial counts in the Global_Cache.

At this tail stage, we leverage a fundamental property of labeled graphs: query vertices with distinct semantic labels must be mapped to data vertices with distinct labels. Consequently, the cached matching counts belonging to different labels are intrinsically orthogonal—they naturally satisfy the global injectivity constraint without any risk of cross-label vertex conflicts. So we can directly compute the valid full matchings by performing a Cartesian product (Line 3) across the cached partial matchings for all labels, without any further enumeration or conflict checks.

Table 2: Statistics of Real-world Data Graphs

Dataset	$\|V\|$	$\|E\|$	d	Domain
yeast	3,101	12,519	8.1	Biological
human	4,674	86,282	36.9	Biological
hprd	9,303	34,998	7.5	Biological
email	36,692	183,831	10.0	Social
wordnet	146,005	656,999	9.0	Lexical
twitch	168,114	6,797,557	80.9	Social
dblp	317,080	1,049,866	6.6	Co-authorship
eu2005	862,664	16,138,468	37.4	Web
patents	3,774,768	16,518,947	8.8	Citation

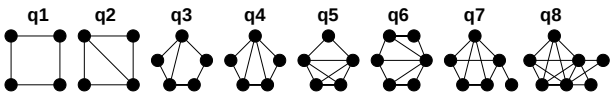


Figure 8: Global queries.

5 EXPERIMENTS

Using real-life and synthetic graphs, we evaluate the effectiveness and efficiency of the proposed method.

Experimental setting. We start with the setting.

Data Graphs. We conduct experiments using both real-world and synthetic graphs. (1) For real-world datasets, we select nine graphs widely used in previous subgraph matching studies [25, 43, 47]. These graphs cover various domains, including biological networks, co-authorship networks, web graphs, and social networks. Their statistics are summarized in Table 2. (2) To evaluate scalability, we generated synthetic graphs using EvoGraph[40], which scales smaller seed graphs while preserving their key structural chareacteristics.

Both real-world and synthetic graphs followed the same label assignment procedure. The label sizes (*i.e.,* the number of vertex labels) were 15, 30, 45, and 60, and vertex labels were randomly assigned following common practice [35, 43].

Query Graphs. We generated query graphs using a sampling method [25]. More specifically, we performed a Metropolis-Hastings

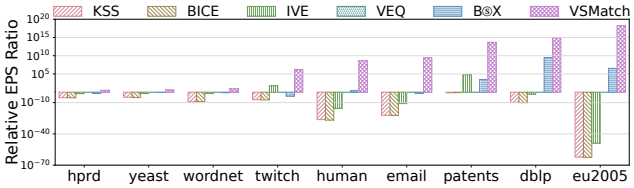


Figure 9: Relative EPS ratio (take VEQ as baseline method).

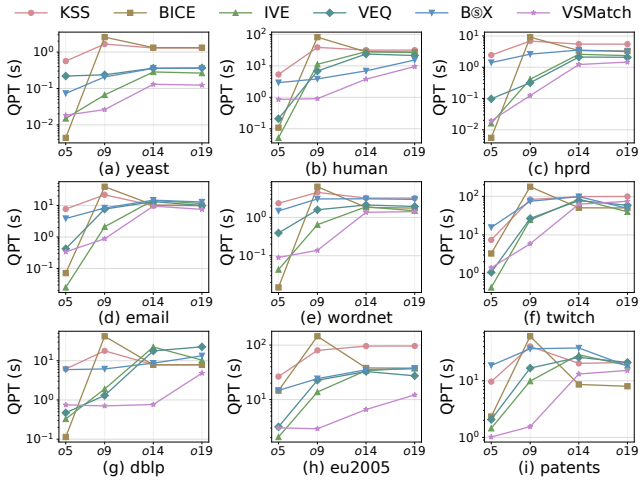


Figure 10: QPT with varying output limits.

random walk[21] on data graphs and extracted induced subgraphs as queries. By default, the query graph size was set to 10, 20, 30, 40, and 50. For each size i, we generated a query set Q_i with 300 queries.

To evaluate the robustness, we also included challenging queries [27] (see Figure 8), referred as global queries [35].

Algorithms. We implemented algorithm VSMatch in C++, and a variant VSMatch-lt, which disable the label trigger mechanism. We compared with five baselines: (1) VEQ [30], using dynamic equivalences for backward-looking local pruning within the backtracking framework; (2) KSS [45], reducing backtracking overhead by decomposing query vertices into conditional kernel and shell structures; (3) BICE [11], compressing the search space by combining global bipartite matching with cell-wide verification; (4) IVE [26], accelerating enumeration by separating isolated query vertices and resolving them via delayed bipartite matching; and (5) B$\circledS X$ [35], grouping vertices into search boxes for batch processing.

For a fair comparison, we used the C++ implementations released by the original authors.

Metrics. We evaluate all methods using three metrics.

(1) Embeddings per second (EPS). EPS measures the average number of embeddings (*i.e.,* subgraphs) generated per second. Following [47], we used EPS as a throughput metric for comparing enumeration performance, incorporating both the number of generated embeddings and the actual elapsed time.

(2) Relative EPS Ratio. Following [35], we report the relative EPS ratio to compare throughput gains. More specifically, the EPS of each method was normalized by the EPS of VEQ[30], which served as the designated baseline. Therefore, the relative EPS ratio of VEQ was fixed to 1, and a larger ratio indicates higher enumeration throughput.

(3) Query processing time (QPT). QPT measures the time required to output a specified number of embeddings. Following [47], we report the time required to generate specified embeddings, with a time limit of 300s for each query.

Experiment Environment. All experiments were conducted on a Linux server running Ubuntu 24.04, with an Intel Xeon Silver 4214 CPU at 2.2 GHz and 256 GB of RAM. Each experiment was repeated three times, and the average is reported.

Experimental results. We next report our findings.

Exp-1: Efficiency. We test the efficiency of VSMatch.

Under the relative EPS metric. We evaluate the throughput of VSMatch using the relative EPS ratio with respect to VEQ. The results are shown in Figure 9, and we find the following.

(1) Algorithm VSMatch achieves the highest relative EPS ratio on all datasets. VSMatch outperforms B$\circledS X$ by 7× on yeast, up to 10^{11}× on eu2005, with an average speedup of 10^6×; similarly, it outperforms VEQ by 3× on hprd, up to 10^{18}× on eu2005, also averaging 10^6×.

(2) On relatively small data graphs yeast and hprd and sparse graph wordnet, the search space is limited, and existing backtracking methods can already complete enumeration efficiently, resulting in relatively small diffferences throughput gap among different methods.

(3) The advantage of VSMatch becomes more pronounced on larger or structurally denser graphs. On graphs excluding hprd, yeast and wordnet, VSMatch beats B$\circledS X$ by 10^5× on dblp, up to 10^{11}× on eu2005, with an average speed up of nearly 10^9 it beats VEQ by 10^6× on twitch, up to 10^{18}× on eu2005, with an average speed up over 10^9×. This is because VSMatch groups structurally equivalent candidates into equivalence classes and enumerates over these compressed units, which substantially reduces redundant exploration.

Under the QPT metric. We next evaluated the robustness using QPT under increasing output limits of 10^5, 10^9, 10^{14} and 10^{19}, denoted as $o5$, $o9$, $o14$ and $o19$, respectively.

The results, as illustrated in Figure 10, reveal that the QPT performance of baseline methods is highly sensitive to the output limit. While these state-of-the-art backtracking algorithms can achieve sub-second runtimes at low limits ($o5$), their execution times grow rapidly as the output cap increases. At higher limits, the sheer volume of redundant branches causes severe performance degradation in traditional methods. In contrast, VSMatch exhibits stable performance scale at higher output limits. This confirms VSMatch's suitability and superiority for massive-output or full-enumeration scenarios in complex, real-world graphs, proving that the initial

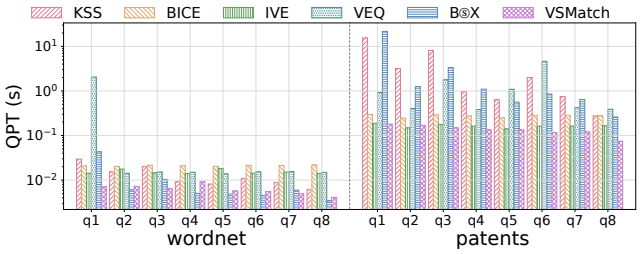


Figure 11: QPT of global queries.

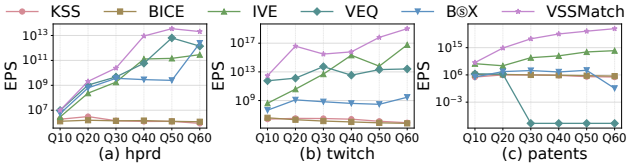


Figure 12: EPS with varying query graph size.

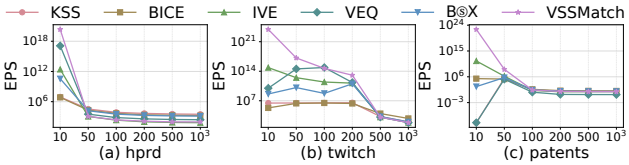


Figure 13: EPS with varying label size.

computational cost of structural filtering yields significant scalability improvements.

Global Query Evaluation. We evaluated the performance of VSMatch using the challenging global queries in Figure 8 on wordnet and patents with an output limit of 10^5 and timeout of 300s.

As shown in Figure 11, we found the following. (1) On small graphs wordnet, most methods finish quickly due to the small search space. (2) On large graph patents, where the search space is much larger, VSMatch outperforms KSS, BICE, IVE, VEQ, B$\circledS$X by reducing the QPT by 96.5%, 50.9%, 17.3%, 89.2%, 96.3% respectively, due to the class-level filtering.

Exp-2: Scalability. We evaluate the scalability of VSMatch by varing quey graph size, label size and data graph size.

Varying the Query Graph Size. We vary the query graph size from 10 to 60 on hprd, twitch, and patents. Figure 12 shows that VSMatch achieves the highest EPS for all query sizes on all three datasets, while showing different trends across datasets. On small hprd, the limited search space reduces the benefit of class-level compression, so the gap to the baselines narrows for larger queries. On dense twitch, similar neighborhood structures help VSMatch form effective equivalence classes, but higher filtering and enumeration cost at Q30 and higher query size offset part of this benefit and cause an EPS drop. On large but relatively sparse patents, many candidate vertices still allow VSMatch to form useful equivalence classes, so its EPS increases as query size increases.

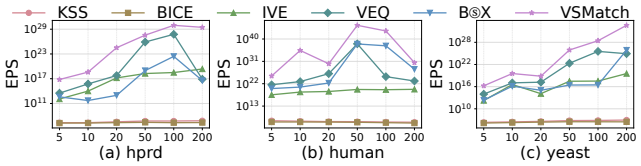


Figure 14: EPS with varying graph size.

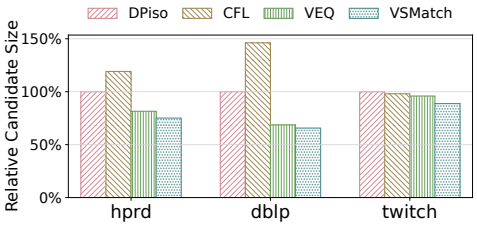
Varying the Label Size. We vary the label size from 10 to 1000 on hprd, twitch, and patents. Figure 13 shows that VSMatch has a clear advantage at small label sizes, since fewer labels produce large candidate sets with more structural similar verticws, making equivalence-class grouping more effective. As the label size increases, candidate sets and embeddings shrink sharply, leaving less redundancy to compress. Therefore, the advantage of VSMatch gradually diminishes as the label size increases, but the turning point differs across datasets. On hprd, VSMatch becomes less competitive from label size 50 because the graph is small and larger label spaces quickly reduce candidate redundancy. On dense twitch, useful equivalence classes can still be formed until larger label sizes; its advantage weakens mainly from label size 500. On patents, although the graph is large, its relatively sparse structure makes candidate sets shrink quickly as labels increase, and the EPS values of different methods become close from label size 100.

Varying the Data Graph Size. We vary the data graph size by scaling the original graphs using EvoGraph [40], which preserves the original topological properties. We scale three real-world datasets (hprd, human, and yeast) by factors of 5, 10, 20, 50, 100, and 200. Figure 14 reports the EPS on these synthetically scaled graphs. On the sparse graph(hprd, yeast), VSMatch demonstrates excellent scalability. As the graph size increases, VSMatch compresses the expanding search space into macroscopic equivalence classes. This effective compression guarantees structural pruning performance, allowing the enumeration throughput (EPS) to scale positively alongside the data volume. Conversely, on the dense human graph (Fig. 14(b)), as the dense topology scales up, the computational overheads associated with macroscopic filtering and set-level enumeration increase significantly. VSMatch's EPS peaks at the 50x scale and subsequently declines at the 100x and 200x scales.

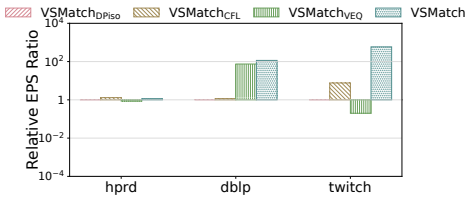
Exp-3: Ablation Study. We conducted ablation studies to evaluate components of VSMatch, including VSSim filtering, enumeration, and label-triggered pre-caching.

Effectiveness of VSSim *Filtering.* We compared VSSim-based filtering with DPiso[19], CFL[7] and VEQ[30] using relative candidate size and relative EPS ratio, both normalized by DPiso. Figure 15(a) shows that VSSim generates the smallest candidate sets on all three datasets, indicating stronger pruning before enumeration. Figure 15(b) further shows that VSSim also achieves the highest relative EPS ratio on all three datasets. On small hprd, the EPS gain is limited since the search space is already small. On larger dblp and denser twitch, richer structural redundancy makes equivalence-classes based filtering more effective, leading to much higher EPS.

Effectiveness of class-level enumeration. To evaluate the pruning capability of VSMatch, we measured its search space size at various

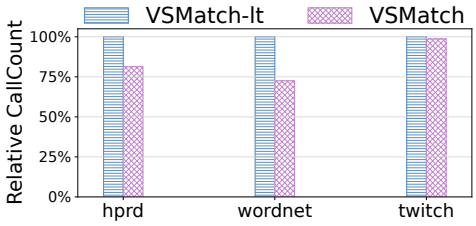


(a) Relative Candidate Size

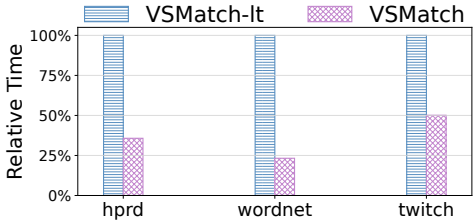


(b) Relative EPS ratio

Figure 15: VSMatch with different filter.



(a) Relative Call Count



(b) Relative time ratio

Figure 16: Effectiveness of Label-Triggered Pre-Caching.

Table 3: Statistic of search space at different depths in the backtracking process.

method&dataset	**depth**	1	3	5	7	9
VSMatch	# search space	1.07	1.13	1.30	0.99	0.00
Genernal	# search space	1.07	1.13	1.28	1.75	3.11
hprd	ratio(%)	100.00	100.00	101.56	56.57	0.00
VSMatch	# search space	1.01	1.03	1.18	0.93	0.0
Genernal	# search space	1.01	1.02	1.10	1.76	4.01
human	ratio(%)	100.00	100.98	100.74	13.70	0.00
VSMatch	# search space	2.09	2.77	4.13	67.00	0
Genernal	# search space	2.28	6.03	32.31	263.08	2450.23
dblp	ratio(%)	91.67	45.94	12.78	1.01	0.00
VSMatch	# search space	33.58	59.69	131.45	240.53	0.00
Genernal	# search space	36.16	78.52	249.13	1222.91	7075.96
patents	ratio(%)	92.87	76.02	52.76	37.14	0.00

search depths and compared it against a standard point-by-point backtracking baseline (General). Using a sample of 100 queries of size 10, we recorded the average node count at depths 1 through 9 in Table 3. On sparse graphs (e.g., hprd and human), VSMatch initially shows a slightly larger search space at shallow depths. This overhead reflects a delayed verification strategy: rather than performing strict conflict checks prematurely, VSMatch postpones fine-grained evaluations to the ComputePartialEmbeddings phase, thereby avoiding unnecessary computation in simple topologies. The true pruning power emerges at deeper levels and on complex graphs (dblp and patents), where the General baseline suffers from severe combinatorial explosions. Remarkably, the search space of VSMatch eliminates the necessity for further node expansions at depth 9 across all datasets. At these depths, the remaining vertices are strictly isolated vertices. Instead of continuing the optimized DFS for exact extraction, VSMatch directly retrieves the final embeddings via a Cartesian product from the Global_Cache, bypassing the tail enumeration.

Effectiveness of Label-Triggered Pre-Caching Mechanism. To evaluate the contribution of the Label-Trigger mechanism, we compare the full VSMatch algorithm against an ablated variant with the trigger disabled (VSMatch-**lt**). We executed 200 random queries under default settings on three datasets (hprd, wordnet, and twitch). Performance is measured using the Relative Call Count and Relative Time, both normalized to VSMatch-**lt**. Here, *Call Count* refers to the total number of recursive function invocations during enumeration, serving as a reliable proxy for the volume of the explored search space. As shown in Figure 16, enabling the label trigger mechanism consistently reduces the call count, confirming its ability to prune infeasible branches early and force early backtracks. While the call count reduction is less pronounced on the twitch dataset, VSMatch still achieves a substantial 50% time reduction (Figure 16b). This reveals a key dual advantage of our design: beyond early pruning, the Label-Trigger also drives the global pre-caching module. By buffering and reusing the computed partial embeddings for isolated vertices, VSMatch eliminates a large portion of redundant local computations across different search branches. These results demonstrate that the Label-Trigger mechanism delivers significant acceleration through the combination of proactive search space pruning and extensive computation reuse.

6 RELATED WORK

6.1 Subgraph Matching

Existing exact subgraph matching algorithms can be broadly classified into two mainstream categories: join-based approaches[29, 34, 38] and backtracking-based approaches[9, 28].

Join-based methods formulate the subgraph isomorphism problem as a multi-way relational join query over edge tables. Grounded in the theoretical breakthroughs of Worst-Case Optimal Joins

(WCOJ) [39], systems like EmptyHeaded [1] and BiGJoin [4] provide rigorous theoretical guarantees on the maximum size of intermediate results. However, despite their theoretical optimality, join-based methods often materialize massive intermediate tables, suffering from severe memory consumption and I/O bottlenecks when dealing with dense, real-world graph topologies.

Consequently, modern in-memory subgraph matching predominantly relies on the backtracking-based paradigm, which avoids massive intermediate materialization by incrementally mapping query vertices to data vertices. The classical algorithm by Ullmann [44] established the foundational depth-first backtracking framework for subgraph isomorphism. Later algorithms like VF2 [12] introduced neighborhood feasibility rules for earlier pruning. To further tame the massive search space, modern algorithms have universally adopted a "filtering-ordering-enumerating" paradigm. Early milestone works such as GraphQL [22] and QuickSI [41] introduced sophisticated structural filtering (e.g., neighborhood signatures) and infrequent-edge-first matching orders, respectively.

Building upon this paradigm, recent state-of-the-art methods introduce advanced data structures and dynamic execution strategies. For instance, TurboISO [20] merges similar query vertices into Neighborhood Equivalence Classes (NEC) to construct a compacted search region. CECI [6] proposes a compact embedding cluster index and utilizes set intersections for fast candidate resolution. CFL-Match [7] decomposes the query graph into core, forest, and leaf structures to strategically delay the Cartesian products of leaves. DPiso [19] utilizes dynamic programming based on Directed Acyclic Graphs (DAG) to generate robust candidate spaces. While these methods profoundly optimize the matching order, candidate sizes, and intersection efficiency, they fundamentally execute candidate enumeration sequentially on a strict vertex-by-vertex basis. This point-by-point exploratory mechanism leaves them highly vulnerable to deep combinatorial explosions and redundant re-evaluations in datasets replete with dense and symmetric topologies.

6.2 Graph Simulation

To circumvent the NP-hardness of exact subgraph isomorphism, numerous simulation models have been proposed as relaxed alternatives. Graph Simulation [15] relaxes the constraint to preserve only downward relationships but fails to capture undirected topologies, often erroneously mapping query cycles to data trees and failing to maintain weak connectivity. Dual Simulation [36] extends this to upward parent relationships to preserve both directed/undirected cycles and weak connectivity. However, it lacks spatial locality, often mapping a connected query graph to completely disconnected components in the data graph and producing excessively large, unmanageable match relations.

Strong Simulation [36] addresses this by enforcing a locality constraint via a bounding ball to guarantee a connected match graph, while Strict simulation [17] and Tight simulation [24] further refine structural size constraints. All these traditional simulations inherently trade topological precision for polynomial-time efficiency by allowing multi-valued, non-injective mappings, which introduces massive false positives with respect to exact subgraph isomorphism.

7 CONCLUSION

This paper presented VSMatch, an equivalence-class-based subgraph matching framework that compresses the search space using vertex-set simulation (VSSim). By grouping locally indistinguishable candidates and performing macroscopic enumeration over equivalence classes, VSMatch drastically reduces the branching factor of backtracking. The label-triggered pre-caching mechanism further accelerates enumeration by handling isolated vertices via early pruning and Cartesian-product reuse. Experiments on real and synthetic graphs show that VSMatch outperforms state-of-the-art methods by up to 11-18 orders of magnitude in EPS throughput.

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at URL_TO_YOUR_ARTIFACTS.

REFERENCES

[1] Christopher R. Aberger, Susan Tu, Kunle Olukotun, and Christopher Ré. 2016. EmptyHeaded: A Relational Engine for Graph Processing. In *SIGMOD*. 431–446.

[2] Wei Ai, CanHao Xie, Tao Meng, Jiayi Du, and Keqin Li. 2024. A D-Truss-Equivalence Based Index for Community Search Over Large Directed Graphs. *TKDE* 36, 11 (2024), 5482–5494.

[3] Waqas Ali, Muhammad Saleem, Bin Yao, Aidan Hogan, and Axel-Cyrille Ngonga Ngomo. 2022. A survey of RDF stores & SPARQL engines for querying knowledge graphs. *VLDBJ* 31, 3 (2022), 1–26.

[4] Khaled Ammar, Frank McSherry, Semih Salihoglu, and Manas Joglekar. 2018. Distributed Evaluation of Subgraph Queries Using Worst-case Optimal and Low-Memory Dataflows. *PVLDB* 11, 6 (2018), 691–704.

[5] Junya Arai, Yasuhiro Fujiwara, and Makoto Onizuka. 2023. GuP: Fast Subgraph Matching by Guard-based Pruning. *Proc. ACM Manag. Data* 1, 2 (2023), 167:1–167:26.

[6] Bibek Bhattarai, Hang Liu, and H. Howie Huang. 2019. CECI: Compact Embedding Cluster Index for Scalable Subgraph Matching. In *SIGMOD*, Peter Boncz, Stefan Manegold, Anastasia Ailamaki, Amol Deshpande, and Tim Kraska (Eds.). 1447–1462.

[7] Fei Bi, Lijun Chang, Xuemin Lin, Lu Qin, and Wenjie Zhang. 2016. Efficient Subgraph Matching by Postponing Cartesian Products. In *SIGMOD*, Fatma Özcan, Georgia Koutrika, and Sam Madden (Eds.). 1199–1214.

[8] Vincenzo Bonnici, Rosalba Giugno, Alfredo Pulvirenti, Dennis E. Shasha, and Alfredo Ferro. 2013. A subgraph isomorphism algorithm and its application to biochemical data. *BMC Bioinform.* 14, S-7 (2013), S13.

[9] Vincenzo Bonnici, Rosalba Giugno, Alfredo Pulvirenti, Dennis E. Shasha, and Alfredo Ferro. 2013. A subgraph isomorphism algorithm and its application to biochemical data. *BMC Bioinform.* 14, S-7 (2013), S13.

[10] Vincenzo Carletti, Pasquale Foggia, Alessia Saggese, and Mario Vento. 2017. Introducing VF3: A New Algorithm for Subgraph Isomorphism. In *GbRPR*. 128–139.

[11] Yunyoung Choi, Kunsoo Park, and Hyunjoon Kim. 2023. BICE: Exploring Compact Search Space by Using Bipartite Matching and Cell-Wide Verification. *PVLDB* 16, 9 (2023), 2186–2198.

[12] Luigi P. Cordella, Pasquale Foggia, Carlo Sansone, and Mario Vento. 2004. A (Sub)Graph Isomorphism Algorithm for Matching Large Graphs. *TPAMI* 26, 10 (2004), 1367–1372.

[13] Kayhan Erciyes. 2023. Graph-Theoretical Analysis of Biological Networks: A Survey. *Comput.* 11, 10 (2023), 188.

[14] Wenfei Fan. 2012. Graph pattern matching revised for social network analysis. In *ICDE*, Alin Deutsch (Ed.). 8–21.

[15] Wenfei Fan, Jianzhong Li, Shuai Ma, Nan Tang, Yinghui Wu, and Yunpeng Wu. 2010. Graph Pattern Matching: From Intractable to Polynomial Time. *PVLDB* 3, 1 (2010), 264–275.

[16] Binbin Fang, Hanzhu Chen, Weiwei Wang, and Yansong Wang. 2024. GraphFA: Graph Enhanced Fraud Detectors with Camouflage Detection for Financial Anti-Fraud. In *ICSP*. 323–327.

[17] Arash Fard, M. Usman Nisar, Lakshmish Ramaswamy, John A. Miller, and Matthew Saltz. 2013. A distributed vertex-centric approach for pattern matching in massive graphs. In *BigData*, Xiaohua Hu, Tsau Young Lin, Vijay V. Raghavan, Benjamin W. Wah, Ricardo Baeza-Yates, Geoffrey C. Fox, Cyrus Shahabi, Matthew Smith, Qiang Yang, Rayid Ghani, Wei Fan, Ronny Lempel, and Raghunath Nambiar (Eds.). 403–411.

[18] M. R. Garey and David S. Johnson. 1979. *Computers and Intractability: A Guide to the Theory of NP-Completeness.* W. H. Freeman.

[19] Myoungji Han, Hyunjoon Kim, Geonmo Gu, Kunsoo Park, and Wook-Shin Han. 2019. Efficient Subgraph Matching: Harmonizing Dynamic Programming, Adaptive Matching Order, and Failing Set Together. In *SIGMOD*. 1429–1446.

[20] Wook-Shin Han, Jinsoo Lee, and Jeong-Hoon Lee. 2013. Turbo$_{\text{iso}}$: towards ultrafast and robust subgraph isomorphism search in large graph databases. In *SIGMOD*, Kenneth A. Ross, Divesh Srivastava, and Dimitris Papadias (Eds.). 337–348.

[21] W. K. Hastings. 1970. Monte Carlo sampling methods using Markov chains and their applications. *Biometrika* 57, 1 (1970), 97–109.

[22] Huahai He and Ambuj K. Singh. 2008. Graphs-at-a-time: query language and access methods for graph databases. In *SIGMOD*, Jason Tsong-Li Wang (Ed.). 405–418.

[23] M.R. Henzinger, T.A. Henzinger, and P.W. Kopke. 1995. Computing simulations on finite and infinite graphs. In *FOCS*.

[24] Arash Jalal Zadeh Fard, M Nisar, John Miller, and Lakshmish Ramaswamy. 2014. Distributed and Scalable Graph Pattern Matching: Models and Algorithms. *International Journal of Big Data* 1 (01 2014), 1–14.

[25] Haolin Jiang, Santosh Pandey, and Hang Liu. 2025. A Comprehensive Survey of Subgraph Matching: [Experiments & Analysis]. *Proc. ACM Manag. Data* 3, 6 (2025), 1–30.

[26] Zite Jiang, Shuai Zhang, Xingzhong Hou, Mengting Yuan, and Haihang You. 2024. IVE: Accelerating Enumeration-Based Subgraph Matching via Exploring Isolated Vertices. In *ICDE*. 4208–4221.

[27] Tatiana Jin, Boyang Li, Yichao Li, Qihui Zhou, Qianli Ma, Yunjian Zhao, Hongzhi Chen, and James Cheng. 2023. Circinus: Fast Redundancy-Reduced Subgraph Matching. *Proc. ACM Manag. Data* 1, 1 (2023), 12:1–12:26.

[28] Alpár Jüttner and Péter Madarasi. 2018. VF2++ - An improved subgraph isomorphism algorithm. *Discret. Appl. Math.* 242 (2018), 69–81.

[29] Hasara Kalumin and Amol Deshpande. 2025. Optimizing Queries with Many-to-Many Joins. In *ICDE*. 3668–3681.

[30] Hyunjoon Kim, Yunyoung Choi, Kunsoo Park, Xuemin Lin, Seok-Hee Hong, and Wook-Shin Han. 2021. Versatile Equivalences: Speeding up Subgraph Query Processing and Subgraph Matching. In *SIGMOD*, Guoliang Li, Zhanhuai Li, Stratos Idreos, and Divesh Srivastava (Eds.). 925–937.

[31] Jinha Kim, Hyungyu Shin, Wook-Shin Han, Sungpack Hong, and Hassan Chafi. 2015. Taming Subgraph Isomorphism for RDF Query Processing. *PVLDB* 8, 11 (2015), 1238–1249.

[32] Jinsoo Lee, Wook-Shin Han, Romans Kasperovics, and Jeong-Hoon Lee. 2012. An In-depth Comparison of Subgraph Isomorphism Algorithms in Graph Databases. *PVLDB* 6, 2 (2012), 133–144.

[33] Jinsoo Lee, Wook-Shin Han, Romans Kasperovics, and Jeong-Hoon Lee. 2012. An In-depth Comparison of Subgraph Isomorphism Algorithms in Graph Databases. *PVLDB* 6, 2 (2012), 133–144.

[34] Mingdao Li, Peng Peng, Zheyuan Hu, Lei Zou, and Zheng Qin. 2024. Variable-Length Path Query Evaluation Based on Worst-Case Optimal Joins. In *ICDE*. 3311–3323.

[35] Yujie Lu, Zhijie Zhang, and Weiguo Zheng. 2025. B$\circledS$X: Subgraph Matching with Batch Backtracking Search. *Proc. ACM Manag. Data* 3, 1 (2025), 15:1–15:27.

[36] Shuai Ma, Yang Cao, Wenfei Fan, Jinpeng Huai, and Tianyu Wo. 2011. Capturing Topology in Graph Pattern Matching. *PVLDB* 5, 4 (2011), 310–321.

[37] Shuai Ma, Yang Cao, Wenfei Fan, Jinpeng Huai, and Tianyu Wo. 2014. Strong simulation: Capturing topology in graph pattern matching. *TODS* 39, 1 (2014), 4:1–4:46.

[38] Amine Mhedhbi and Semih Salihoglu. 2019. Optimizing Subgraph Queries by Combining Binary and Worst-Case Optimal Joins. *PVLDB* 12, 11 (2019), 1692–1704.

[39] Hung Q. Ngo, Ely Porat, Christopher Ré, and Atri Rudra. 2018. Worst-case Optimal Join Algorithms. *JACM* 65, 3 (2018), 16:1–16:40.

[40] Himchan Park and Min-Soo Kim. 2018. EvoGraph: An Effective and Efficient Graph Upscaling Method for Preserving Graph Properties. In *KDD*, Yike Guo and Faisal Farooq (Eds.). 2051–2059.

[41] Haichuan Shang, Ying Zhang, Xuemin Lin, and Jeffrey Xu Yu. 2008. Taming verification hardness: an efficient algorithm for testing subgraph isomorphism. *PVLDB* 1, 1 (2008), 364–375.

[42] Junshuai Song, Xiaoru Qu, Zehong Hu, Zhao Li, Jun Gao, and Ji Zhang. 2021. A subgraph-based knowledge reasoning method for collective fraud detection in E-commerce. *Neurocomputing* 461 (2021), 587–597.

[43] Shixuan Sun and Qiong Luo. 2020. In-Memory Subgraph Matching: An In-depth Study. In *SIGMOD*, David Maier, Rachel Pottinger, AnHai Doan, Wang-Chiew Tan, Abdussalam Alawini, and Hung Q. Ngo (Eds.). 1083–1098.

[44] Julian R. Ullmann. 1976. An Algorithm for Subgraph Isomorphism. *JACM* 23, 1 (1976), 31–42.

[45] Rongjian Yang, Zhijie Zhang, Weiguo Zheng, and Jeffrey Xu Yu. 2023. Fast Continuous Subgraph Matching over Streaming Graphs via Backtracking Reduction. *Proc. ACM Manag. Data* 1, 1 (2023), 15:1–15:26.

[46] Qi Zhang, Rong-Hua Li, Hongchao Qin, Guoren Wang, Zhiwei Zhang, and Ye Yuan. 2023. Stable Subgraph Isomorphism Search in Temporal Networks. *TKDE* 35, 6 (2023), 6405–6420.

[47] Zhijie Zhang, Yujie Lu, Weiguo Zheng, and Xuemin Lin. 2024. A Comprehensive Survey and Experimental Study of Subgraph Matching: Trends, Unbiasedness, and Interaction. *Proc. ACM Manag. Data* 2, 1 (2024), 60:1–60:29.